Unit I: Introduction to Data and Database

This unit lays the groundwork for understanding databases.

- **Introduction to Data and Database:**
    - **Data:** Raw facts and figures, often unstructured, that have no inherent meaning until organized. Examples: a list of names, numbers, dates.
    - **Information:** Processed, organized, and structured data that provides context and meaning. Information is derived from data. Examples: a report showing sales trends, a customer's complete profile.
    - **Database:** An organized collection of structured information, or data, typically stored electronically in a computer system. It's designed for efficient storage, retrieval, and management of large amounts of data.
    - **Evolution of Databases:** Briefly touch upon traditional file processing systems and their limitations, leading to the need for database systems.
- **Significance of Database Management System (DBMS):**
    - **What is a DBMS?** Software that allows users to define, create, maintain, and control access to the database. It acts as an interface between the database and the end-users or application programs.
    - **Advantages/Significance:**
        - **Data Redundancy Control:** Minimizes duplication of data, saving storage space and improving data consistency.
        - **Data Inconsistency Control:** Ensures that the same data doesn't have conflicting values in different places.
        - **Data Sharing:** Allows multiple users and applications to access the same data concurrently.
        - **Data Security:** Provides mechanisms for access control and protection against unauthorized access.
        - **Data Integrity:** Enforces rules and constraints to ensure the accuracy and reliability of data.
        - **Data Independence:** Allows changes to the database schema at one level without affecting other levels (see below).
        - **Backup and Recovery:** Provides tools to recover data in case of system failures.
        - **Concurrency Control:** Manages simultaneous access by multiple users to prevent data corruption.
        - **Improved Data Access:** Facilitates quick and efficient retrieval of data through query languages.

        - **Reduced Application Development Time:** DBMS handles many low-level data management tasks, allowing developers to focus on application logic.

- **Various Types of DBMS:**
    - **Hierarchical DBMS:** Data is organized in a tree-like structure with a one-to-many relationship (parent-child). (e.g., IBM IMS)
    - **Network DBMS:** Similar to hierarchical but allows more complex relationships, enabling a record to have multiple parents (many-to-many).
    - **Relational DBMS (RDBMS):** The most common type. Data is organized into tables (relations) with rows and columns. Relationships are established through common fields. (e.g., MySQL, PostgreSQL, Oracle, SQL Server).
    - **Object-Oriented DBMS (OODBMS):** Data is stored as objects, similar to objects in object-oriented programming. (e.g., db4o)
    - **NoSQL Databases:** A broad category of databases that don't adhere to the traditional relational model. Designed for specific use cases, often dealing with large volumes of unstructured or semi-structured data. (e.g., MongoDB, Cassandra, Redis).
        - Briefly mention types like Document, Key-Value, Column-Family, Graph.
- **Data Independence - The Three Levels of Architecture (ANSI/SPARC Architecture):**
    - **Data Independence:** The ability to modify the schema at one level without affecting the schema at a higher level.
        - **Physical Data Independence:** Changes to the internal schema (e.g., file organization, indexing) do not require changes to the conceptual or external schemas.
        - **Logical Data Independence:** Changes to the conceptual schema (e.g., adding/removing attributes or entities) do not require changes to the external schemas or application programs.
    - **The External Level (View Level):**
        - Describes the part of the database relevant to a particular user or application.
        - Presents a simplified and customized view of the data, hiding irrelevant details.
        - Multiple external schemas can exist for a single conceptual schema.
    - **Conceptual Level (Logical Level):**
        - Describes the entire database structure from the perspective of the organization.
        - Defines entities, attributes, relationships, data types, and constraints.
        - Independent of physical storage details.
        - Acts as a bridge between the external and internal levels.

    - **Internal Level (Physical Level):**
        - Describes the physical storage structure of the database.
        - Details how data is actually stored on disk (e.g., file organization, indexing, data compression).
        - Closest to the physical storage media.
- **Client/Server Architecture:**
    - A common architectural model for DBMS.
    - **Client:** An application or user interface that requests services from the database server.
    - **Server:** The DBMS software running on a separate machine that manages the database and provides services to clients.
    - **Advantages:** Centralized control, easier maintenance, scalability, enhanced security.
- **System Structure:**
    - Refers to the overall components and their interactions within a database system (e.g., query processor, storage manager, transaction manager, buffer manager).
- **Instance and Schema:**
    - **Schema:** The description of the database. It defines the structure, organization, and relationships of the data. It's the "blueprint" or "design" of the database. (e.g., table definitions, data types, constraints).
        - **Conceptual Schema:** Overall logical design.
        - **Internal Schema:** Physical storage details.
        - **External Schemas:** User-specific views.
    - **Instance:** The actual content or data stored in the database at a particular point in time. It's the "snapshot" of the database. The instance changes frequently as data is added, deleted, or modified, while the schema generally remains static.

Unit II: Keys, ER Model, and Basic SQL

This unit focuses on foundational database design concepts and the basics of SQL.

- **Keys:**
    - **Primary Key:** A column or a set of columns that uniquely identifies each row in a table. Must be unique and not null.
    - **Candidate Key:** Any column or set of columns that can uniquely identify a row in a table. A primary key is chosen from the candidate keys.
    - **Super Key:** Any set of attributes that uniquely identifies a tuple (row) in a relation. Includes candidate keys and supersets of candidate keys.

- **Foreign Key:** A column or a set of columns in one table that refers to the primary key of another table. Used to establish relationships between tables.
- **Composite Key:** A key that consists of two or more attributes.
- **Alternate Key:** A candidate key that is not chosen as the primary key.

- **CODD's Rules:**
  - Edgar F. Codd proposed 12 rules (though often stated as 13, with Rule 0) that a database must satisfy to be considered truly relational. These rules define the characteristics of a relational database system. Key rules include:
    - **Rule 0: The foundation rule:** All information is represented as values in tables.
    - **Rule 1: Information Rule:** All information (data and metadata) is represented in tables.
    - **Rule 2: Guaranteed Access Rule:** Every data item must be logically accessible by using its table name, primary key value, and column name.
    - **Rule 3: Systematic Treatment of Null Values:** Proper handling of null values (representing missing or inapplicable information).
    - **Rule 4: Dynamic Online Catalog based on the Relational Model:** The database description (schema) is stored in the database itself.
    - **Rule 5: Comprehensive Data Sublanguage Rule:** A single, well-defined language (like SQL) supports data definition, manipulation, and control.
    - **Rule 6: View Updating Rule:** All views that are theoretically updatable must be updatable by the system.
    - **Rule 7: High-Level Insert, Update, and Delete:** The system must support set-at-a-time operations.
    - **Rule 8: Physical Data Independence.**
    - **Rule 9: Logical Data Independence.**
    - **Rule 10: Integrity Independence:** Integrity constraints should be defined in the relational language and stored in the catalog.
    - **Rule 11: Distribution Independence:** The database should work correctly even if distributed across multiple locations.
    - **Rule 12: Non-subversion Rule:** No way to bypass the integrity rules using a lower-level language.

- **Design Issues - ER – Model:**
  - **ER Model (Entity-Relationship Model):** A high-level conceptual data model used to design a database. It represents real-world entities and their relationships.
  - **Entities:** Real-world objects or concepts about which data is stored. (e.g., Student, Course, Department). Represented by rectangles.

        - **Entity Set:** A collection of similar entities.
    - **Attributes:** Properties or characteristics of an entity. (e.g., for Student: Student ID, Name, Age). Represented by ovals.
        - **Attribute Types:**
            - **Simple Attribute:** Cannot be further divided (e.g., Age).
            - **Composite Attribute:** Can be divided into smaller sub-parts (e.g., Name can be divided into First Name, Middle Name, Last Name).
            - **Single-valued Attribute:** Has only one value for an entity (e.g., Student ID).
            - **Multi-valued Attribute:** Can have multiple values for an entity (e.g., Phone Numbers for a Student). Represented by double ovals.
            - **Derived Attribute:** Can be derived from other attributes (e.g., Age from Date of Birth). Represented by dashed ovals.
            - **Key Attribute:** An attribute that uniquely identifies an entity (e.g., Student ID). Underlined.
    - **Relationships:** Associations between entities. (e.g., Student *enrolls in* Course). Represented by diamonds.
        - **Relationship Types:**
            - **One-to-One (1:1):** Each entity in one set is related to at most one entity in another set.
            - **One-to-Many (1:M):** One entity in the first set can be related to multiple entities in the second set, but an entity in the second set is related to at most one in the first.
            - **Many-to-One (M:1):** Multiple entities in the first set can be related to one entity in the second set, but an entity in the second set is related to at most one in the first.
            - **Many-to-Many (M:N):** Multiple entities in the first set can be related to multiple entities in the second set.
        - **Degree of a Relationship:** The number of entity sets participating in a relationship (binary, ternary, etc.).
    - **Weak Entity Sets:**
        - An entity set that does not have sufficient attributes to form a primary key on its own.
        - Its existence is dependent on another entity set (the "identifying owner" or "strong entity").
        - Represented by a double rectangle.

        - The relationship connecting a weak entity set to its strong entity set is an "identifying relationship" (double diamond).
        - Its primary key is formed by the primary key of the strong entity combined with its partial key.

- **Extended ER Features (EER Model):**
    - **Specialization:** Top-down design process where an entity set is divided into specialized sub-entity sets (e.g., Person -> Employee, Student).
    - **Generalization:** Bottom-up design process where common properties of multiple entity sets are combined into a higher-level entity set (e.g., Employee, Student -> Person).
    - **Aggregation:** Represents a relationship as a higher-level entity. Used when a relationship itself participates in another relationship.
    - **Composition/Association:** Not standard EER but can be mentioned as a type of relationship indicating a "part-of" relationship.
    - **Inheritance:** Sub-entities inherit attributes and relationships from their super-entities.
- **ER to Relational Mapping:**
    - The process of converting an ER model into a relational database schema (tables, columns, primary keys, foreign keys).
    - **Mapping Entities:** Each strong entity set becomes a table. Attributes become columns. The primary key of the entity becomes the primary key of the table.
    - **Mapping Weak Entities:** A weak entity set becomes a table. Its primary key is the combination of its partial key and the primary key of its identifying owner (which becomes a foreign key).
    - **Mapping One-to-One Relationships:** Either primary key can be placed as a foreign key in the other table, or a merged table can be created.
    - **Mapping One-to-Many/Many-to-One Relationships:** The primary key of the "one" side is placed as a foreign key in the "many" side table.
    - **Mapping Many-to-Many Relationships:** A new intersection table is created. This table will have composite primary key consisting of the primary keys of both participating entities (which also act as foreign keys).
    - **Mapping Multi-valued Attributes:** Create a new table for the multi-valued attribute, with the primary key of the original entity as a foreign key in the new table.
    - **Mapping Composite Attributes:** The simple component attributes become separate columns.
    - **Mapping Derived Attributes:** Not typically stored directly; they are calculated when needed.
- **Structure Of Relational Databases:**

- **Relation (Table):** A set of tuples (rows) and attributes (columns).
- **Tuple (Row):** A single record in a table, representing a specific entity.
- **Attribute (Column):** A named property of a relation, representing a characteristic of the entities.
- **Domain:** The set of permissible values for an attribute.
- **Schema:** The logical design of the relation (table name, column names, data types).
- **Instance:** The actual data in the table at a given time.

- **Creation and Manipulation of Database using Basic SQL (DDL, DML, DCL, TCL):**
  - **SQL (Structured Query Language):** The standard language for managing and querying relational databases.
  - **DDL (Data Definition Language):** Used to define and manage the database schema.
    - CREATE DATABASE: Create a new database.
    - CREATE TABLE: Define a new table (columns, data types, constraints).
    - ALTER TABLE: Modify an existing table structure (add/drop columns, change data types).
    - DROP TABLE: Delete a table.
    - TRUNCATE TABLE: Remove all rows from a table, but keep its structure.
  - **DML (Data Manipulation Language):** Used to manage data within the database.
    - INSERT INTO: Add new rows of data.
    - SELECT: Retrieve data from tables.
      - Basic SELECT with FROM, WHERE, ORDER BY, GROUP BY, HAVING.
      - Joins (INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL OUTER JOIN).
      - Set Operations (UNION, INTERSECT, EXCEPT).
    - UPDATE: Modify existing data in rows.
    - DELETE FROM: Remove rows of data.
  - **DCL (Data Control Language):** Used to control access and permissions.
    - GRANT: Give users permissions.
    - REVOKE: Remove users' permissions.
  - **TCL (Transaction Control Language):** Used to manage transactions.
    - COMMIT: Make permanent changes made during a transaction.
    - ROLLBACK: Undo changes made during a transaction.
    - SAVEPOINT: Set a point within a transaction to which you can later roll back.

Unit III: Normalization

This unit delves into the process of organizing database tables to minimize data redundancy and improve data integrity.

- **Normalization:**
    - A systematic process of organizing the columns and tables of a relational database to minimize data redundancy and improve data integrity.
    - Aims to eliminate undesirable characteristics like Insertion, Update, and Deletion Anomalies.
    - Involves decomposing (splitting) tables into smaller, well-structured tables.
- **Anomalies:**
    - Problems that can arise in poorly designed relational databases due to data redundancy.
    - **Insertion Anomaly:** Cannot insert certain data into the database unless other unrelated data is also available. (e.g., Cannot add a new department until an employee is assigned to it).
    - **Update Anomaly:** Updating a piece of data requires updating multiple rows, leading to inconsistency if some updates are missed. (e.g., If a department's name changes, it must be updated for every employee in that department).
    - **Deletion Anomaly:** Deleting a record inadvertently deletes other important data. (e.g., Deleting the last employee from a department might also delete the department's details if they are stored in the same row).
- **Functional Dependency:**
    - A constraint between two sets of attributes in a relation from a database.
    - Given a relation R, attribute B is functionally dependent on attribute A (denoted as $A \rightarrow B$) if for every valid instance of A, that A value uniquely determines the B value. In other words, if two tuples have the same A value, they must also have the same B value.
    - **Trivial Functional Dependency:** $A \rightarrow B$ is trivial if B is a subset of A. (e.g., $\{A,B\} \rightarrow B$).
    - **Non-trivial Functional Dependency:** $A \rightarrow B$ is non-trivial if B is not a subset of A.
- **Armstrong's Axioms:**
    - A set of inference rules used to derive all possible functional dependencies from a given set of functional dependencies.
    - **Reflexivity:** If B is a subset of A, then $A \rightarrow B$. (If you know A, you know B).
    - **Augmentation:** If $A \rightarrow B$, then $AC \rightarrow BC$ (where C is any set of attributes). (You can add attributes to the left side).
    - **Transitivity:** If $A \rightarrow B$ and $B \rightarrow C$, then $A \rightarrow C$. (If A determines B, and B determines C, then A determines C).

- **Additional Derived Rules (not axioms but derivable):**
    - **Decomposition/Projectivity:** If A→BC, then A→B and A→C.
    - **Union/Additivity:** If A→B and A→C, then A→BC.
    - **Pseudotransitivity:** If A→B and CB→D, then AC→D.

- **Closure of a Relation and Closure of Attribute:**
    - **Closure of a set of Functional Dependencies (F+):** The set of all functional dependencies that can be logically inferred from a given set of functional dependencies F, using Armstrong's Axioms.
    - **Closure of an Attribute Set (X+):** The set of all attributes that are functionally determined by a given set of attributes X, based on the functional dependencies in F. This is used to find candidate keys.

- **Lossless Decomposition:**
    - A property of decomposition where the original relation can be reconstructed (joined back) from its decomposed relations without generating any spurious (incorrect or extra) tuples.
    - Ensures that no information is lost during decomposition.

- **Normal Forms:**
    - A series of guidelines used to minimize data redundancy and improve data integrity in a relational database. Each normal form builds upon the previous one.
    - **1NF (First Normal Form):**
        - All attributes must contain atomic (indivisible) values. No multi-valued attributes.
        - Each column must have a unique name.
        - The order of data does not matter.
        - No repeating groups.
    - **2NF (Second Normal Form):**
        - Must be in 1NF.
        - All non-key attributes must be fully functionally dependent on the primary key. (Eliminates partial dependencies, where a non-key attribute depends on only part of a composite primary key).
    - **3NF (Third Normal Form):**
        - Must be in 2NF.
        - No transitive dependencies exist. (Eliminates transitive dependencies, where a non-key attribute depends on another non-key attribute).
    - **Boyce-Codd Normal Form (BCNF):**

        - A stricter version of 3NF.
        - For every non-trivial functional dependency X→Y, X must be a superkey.
        - Eliminates all redundancies based on functional dependencies.
        - Often preferred over 3NF, but sometimes 3NF is chosen if BCNF decomposition is not dependency-preserving (i.e., some functional dependencies cannot be inferred from the decomposed relations).

Unit IV: Relational Algebra, Query Processing, and Transaction Management

This unit covers the theoretical foundation of relational databases, how queries are executed, and how transactions are managed.

- **The Relational Algebra:**
    - A procedural query language that defines a set of operations to manipulate relations (tables).
    - Each operation takes one or two relations as input and produces a new relation as output.
    - **Basic Operations:**
        - **Select (σ):** Filters rows (tuples) based on a given condition. (σcondition(R))
        - **Project (π):** Selects specific columns (attributes) from a relation. (πattributes (R))
        - **Union (∪):** Combines rows from two relations. Relations must be union-compatible (same number and type of attributes).
        - **Set Difference ($-**
          ):Returnsrowspresentinthefirstrelationbutnotinthesecond.Relationsmust beunion−compatible.***CartesianProduct(\times$):** Combines every row from the first relation with every row from the second relation. Produces a new relation with combined attributes. (R×S)
        - **Rename (ρ):** Changes the name of a relation or an attribute. (ρnew_name(R) or ρnew_attribute_name←old_attribute_name(R))
    - **Derived Operations:**
        - **Join (⋈):** Combines related tuples from two relations based on a common attribute or condition.
            - **Theta Join (⋈θ):** Join based on an arbitrary condition.
            - **Equijoin (⋈=):** Join based on equality condition.
            - **Natural Join (⋈):** An equijoin on all common attributes, with duplicate common attributes projected out.
        - **Intersection (∩):** Returns rows common to both relations. Relations must be union-compatible.

        - **Division (÷):** Used to find entities that are related to all entities in another set. (Less common, but important for complex queries).

- **Query Processing and Optimization:**
    - **Query Processing:** The series of steps involved in executing a query and retrieving the requested data.
        - **Parsing and Translation:** The SQL query is parsed into a syntax tree and translated into an equivalent relational algebra expression.
        - **Optimization:** The relational algebra expression is optimized to find the most efficient execution plan.
        - **Evaluation:** The optimized query plan is executed by the query execution engine.
    - **Query Optimization:** The process of finding the most efficient way to execute a given query. Aims to minimize execution time and resource consumption.
        - **Heuristics-based Optimization:** Uses rules of thumb (e.g., performing selections and projections as early as possible).
        - **Cost-based Optimization:** Estimates the cost (I/O, CPU time) of different execution plans and chooses the one with the lowest cost.
            - **Cost factors:** Disk I/O, CPU usage, network communication.
            - **Statistics:** DBMS uses statistics about data (e.g., number of tuples, distinct values, index information) to estimate costs.
    - **Evaluation of Relational Algebra Expressions:**
        - The process of executing the sequence of relational algebra operations defined by the query plan.
        - Involves fetching data from disk, performing operations in memory, and writing results.
    - **Query Equivalence:**
        - Two relational algebra expressions are query equivalent if they produce the same result for any valid database instance.
        - Query optimizers use equivalence rules (e.g., commutativity, associativity of operations, pushing down selections) to transform a query into an equivalent but more efficient form.

- **Transaction Processing:**
    - **Transaction:** A single logical unit of work that accesses and possibly modifies the contents of a database. It's a sequence of operations (read, write) treated as a single, indivisible unit.
    - **ACID Properties:** Fundamental properties that ensure data integrity and reliability in a transactional database system.

        - **Atomicity:** A transaction is an "all-or-nothing" proposition. Either all operations within the transaction are completed successfully, or none are. If any part fails, the entire transaction is rolled back to its initial state.
        - **Consistency:** A transaction brings the database from one valid state to another valid state. It must obey all defined integrity constraints (e.g., primary key, foreign key, not null).
        - **Isolation:** The execution of concurrent transactions should be isolated from each other. The effects of one transaction should not be visible to other concurrent transactions until the first transaction is committed. It appears as if transactions are executed sequentially.
        - **Durability:** Once a transaction is committed, its changes are permanent and survive any subsequent system failures (e.g., power outages, crashes).
- **States of a Transaction:**
    - **Active:** The initial state; the transaction is executing.
    - **Partially Committed:** After the last operation of the transaction has been executed. Before changes are written to disk.
    - **Failed:** The transaction cannot proceed normally due to an error (e.g., logical error, system crash, deadlock).
    - **Aborted:** The transaction has been rolled back, and the database has been restored to its state before the transaction began.
    - **Committed:** The transaction has successfully completed, and its changes have been permanently recorded in the database.
- **Introduction to Concurrency Control:**
    - **Concurrency:** Multiple transactions executing simultaneously.
    - **Concurrency Control:** The set of mechanisms used to ensure that concurrent execution of transactions does not lead to inconsistencies in the database.
    - **Problems with Concurrency:**
        - **Lost Update Problem:** One transaction overwrites the changes of another committed transaction.
        - **Dirty Read Problem (Uncommitted Dependency):** A transaction reads data written by another uncommitted transaction. If the second transaction aborts, the first transaction has read "dirty" data.
        - **Non-repeatable Read Problem:** A transaction reads the same data item twice and gets different values because another committed transaction modified it between the reads.
        - **Phantom Read Problem:** A transaction rereads a range of records and finds that a new record has been inserted into the range by another committed transaction.

- **Concurrency Control Techniques:**
    - **Locking:** Transactions acquire locks on data items before accessing them.
        - **Shared Lock (Read Lock):** Allows multiple transactions to read concurrently.
        - **Exclusive Lock (Write Lock):** Only one transaction can hold an exclusive lock at a time.
        - **Two-Phase Locking (2PL):** A protocol that ensures serializability by requiring transactions to acquire all locks before releasing any, and then release all locks after acquiring none. (Growing phase, Shrinking phase).
    - **Timestamping:** Each transaction is assigned a unique timestamp. Operations are ordered based on timestamps.
    - **Optimistic Concurrency Control:** Transactions proceed without locking, then validate changes at commit time. If conflicts are detected, the transaction is rolled back.

- **Deadlock:**
    - A situation where two or more transactions are indefinitely waiting for each other to release the locks that each holds.
    - **Conditions for Deadlock:** Mutual exclusion, hold and wait, no preemption, circular wait.
    - **Deadlock Handling Strategies:**
        - **Deadlock Prevention:** Design protocols to prevent any of the deadlock conditions from occurring (e.g., acquire all locks at once, ordered locking).
        - **Deadlock Detection and Recovery:** Allow deadlocks to occur, detect them, and then recover by aborting one or more transactions (e.g., using a wait-for graph, victim selection).
        - **Deadlock Avoidance:** Transactions declare their resource needs in advance, and the system dynamically checks if granting a request would lead to a deadlock (e.g., Banker's Algorithm).

- **Recovery:**
    - The process of restoring the database to a consistent state after a system failure (e.g., power outage, disk crash, software error).
    - **Types of Failures:** Transaction failure, system crash, media failure.
    - **Recovery Techniques:**
        - **Log-Based Recovery (Write-Ahead Logging - WAL):** A log file records all database modifications.

        - **Redo:** Reapplying changes from the log for committed transactions that didn't make it to disk.
        - **Undo:** Rolling back changes for aborted or uncommitted transactions.
- **Checkpoints:** Periodically, the DBMS takes a checkpoint, writing all dirty pages (modified data in buffer) to disk and updating the log. This reduces the amount of work needed during recovery.
- **Shadow Paging:** Creates copies of pages for updates; if a transaction aborts, the old pages are retained.
- **Database Backup and Restore:** Full backups are used for media failures to restore the database to a previous consistent state. Transaction logs are then applied to bring it to the latest possible consistent state.